

operations on pools.

Now, for creating a snapshot of any filesystem, we can issue:

```
zfs snapshot K7@snap1
```

The snapshot of a filesystem is represented by its name followed by '@' and then the snapshot name. Use the `-r` option to create snapshots recursively on all filesystems under the specified filesystem, as in what's shown below:

```
zfs snapshot -r K7@snap2
```

Now, after a lot of changes to the filesystem, if you want to go back to a snapshot of the filesystem, issue the `rollback` command. The `-r` switch is required, as we have to remove the newer snapshot 'snap2' to roll back to 'snap1'.

```
zfs rollback -r K7@snap1
```

Or if the snapshot you are rolling back is the newest of all the snapshots of the filesystem, then use the following:

```
zfs rollback K7@snap2
```

As in the listing of pools, datasets (which include filesystems and snapshots) can be displayed using the command given below:

```
zfs list
```

The snapshot created can be easily transferred between pools or even between systems using the commands `send` and `recv`. The following command will create a new filesystem 'K7_snap' under 'K7' from the snapshot 'snap1':

```
zfs send K7@snap1 | zfs recv K7/K7_snap
```

The following command is the same as the one above, but the new filesystem and snapshot will be in a remote system 'sreejith':

```
zfs send K7@snap1 | ssh root@sreejith zfs recv K7/K7_snap
```

As we know, ZFS is the native filesystem of Solaris and if we want to migrate any pool storage in Solaris to some other OS like Linux, then we'll have to first export the pool from Solaris or whatever OS in which it was being used, and then import it to the required OS.

```
zpool export K7
```

In order to forcefully export 'K7', we can use the `-f`

switch with the above command.

The following command will display all importable pools with their name and ID:

```
zpool import
```

...and we can import it using the name (or even the ID), issuing the command below:

```
zpool import K7
```

And finally, the `destroy` command is used to destroy a pool or a filesystem. The following destroys the 'ajc' filesystem in 'K7':

```
zfs destroy K7/ajc
```

...while the next command destroys the K7 pool altogether:

```
zpool destroy K7
```

Though ZFS on FUSE manages to implement a lot of the features of native ZFS, it is still not complete, as has been pointed out in the status of the project. Since the implementation is in userspace, which has to be linked to the Linux kernel through the FUSE module, the performance and scalability is not at par with the kernel module implementation of other filesystems as of version 0.5. Even then, the project is a nice way to get acquainted with the revolutionary ZFS in operating systems like Linux. However, it is expected that a properly tuned ZFS on FUSE may have a comparable performance to the native filesystems as in the case of NTFS-3G, a freely and commercially available and supported fast handling read/write NTFS driver for Linux, FreeBSD, MacOS, etc. 

References

- www.openSolaris.org/os/community/zfs/
- www.wizy.org/
- kerneltrap.org/node/8066
- blogs.sun.com/bonwick/en_US/category/ZFS
- www.Solarisinternals.com/wiki/index.php/ZFS_Best_Practices_Guide
- fuse.sourceforge.net/
- www.csamuel.org/2007/04/25/comparing-ntfs-3g-to-zfs-fuse-for-fuse-performance
- blogs.sun.com/bonwick/entry/128_bit_storage_are_you

By: Anoop Joe Cyriac

He is currently working at K7 Computing, an information security solutions company. Hacking in GNU/Linux using C and Python is his passion. His areas of interest include virtualisation technologies and cloud computing. You can reach him at anoopjoecyriac@gmail.com.